

INFORME TÉCNICO - SESIÓN 09

Capacidades nativas con expect/actual en PharmaMobile_Practica

ESTUDIANTE	Guillen Mendoza Alexander	DOCENTE	David Reyna
CURSO	Desarrollo de Aplicaciones Móviles	FECHA	06 de octubre de 2026
PROYECTO	PharmaMobile_Practica (KMP)	RAMA	feature/expect-actual-guillen

1. Objetivo y alcance

Implementar dos capacidades específicas de plataforma sin introducir dependencias de Android o UIKit en la presentación común: formato de moneda peruana y uso del mecanismo nativo para compartir productos. La solución conserva la separación Clean + MVVM y usa Koin para seleccionar la implementación de cada sistema operativo.

El detalle se conectó al inventario local del proyecto. La guía presupone el CRUD de PharmaSoft de la sesión 8, pero ese backend no está integrado en esta rama; el catálogo REST actual es independiente y no proporciona stock. No se atribuyen a esta entrega el CRUD remoto, JWT ni persistencia offline-first.

2. Implementación realizada

CAPACIDAD	CÓDIGO COMÚN	ANDROID / iOS
Moneda	Formato.kt declara expect fun formatearSoles(Double). ProductoUiState.kt aplica el formato al mapear el modelo a UI.	Formato.android.kt: NumberFormat(es-PE). Formato.ios.kt: NSNumberFormatter(es_PE).
Compartir	Compartidor.kt define el contrato. TextoParaCompartir.kt construye nombre, precio y stock.	CompartidorAndroid.kt: ACTION_SEND y NEW_TASK. CompartidorIos.kt: UIActivityViewController.
Integración	DetalleProductoViewModel recibe Compartidor; DetalleProductoScreen muestra el botón. App.kt navega al detalle.	Ambos PlatformModule registran la implementación correspondiente en Koin.

3. Recorrido de la función

Productos → Detalle → Compartir. La pantalla de productos envía el ID; el ViewModel obtiene el producto desde ProductoRepository y crea ProductoUi con el precio local. Al pulsar Compartir, el ViewModel produce el texto en código común y llama al contrato Compartidor. Koin aporta la clase nativa apropiada. El composable solo dispara el evento y no conoce Intent ni UIKit.

4. Verificación efectuada

Comando ejecutado: ./gradlew.bat :shared:testAndroidHostTest :androidApp:assembleDebug
:shared:compileKotlinIosSimulatorArm64 --offline --console=plain. Resultado: BUILD SUCCESSFUL. Las pruebas de host Android incluyen dos pruebas nuevas sobre el formato de ProductoUi y el texto compartido. La búsqueda de importaciones android.* y platform.UIKit.* dentro de presentation no encontró resultados.

Alcance de la verificación: compilar el código Kotlin iOS no equivale a ejecutar iosApp en Xcode. Tampoco se verificó visualmente la hoja de compartir: el emulador Android detectado no terminó de arrancar durante la comprobación y en Windows no hay simulador iOS.

5. Lista de cotejo de la guía

N.º	CRITERIO	ESTADO HONESTO
1-2	expect y actual con igual paquete y firma	Verificado en código y compilación
3	Precio en soles en Android e iOS	Compila; falta inspección visual en ambos
4	Formateo en mapeo de presentación	Verificado en código
5-6	Contrato de dominio y dos implementaciones nativas	Verificado en código y compilación
7	Registros en ambos platformModule	Verificado en código; falta ejecución iOS
8	Botón Compartir y texto común	Implementado; falta prueba visual en ambos
9	Sin imports nativos en presentation	Verificado por búsqueda
10	Capturas de la misma funcionalidad	Pendiente

6. Entregables y pendientes

Disponibles: rama pública con cinco commits propios y README con la sección Capacidades nativas. Pendientes: captura del listado Android, captura del selector Android, captura del listado iOS, captura de la hoja iOS y captura del error temporal de compilación sin actual. La ausencia de esas imágenes impide presentar la evidencia visual exigida por la guía como completa.

Rama: github.com/alexgm-A11/PharmaMobile_Practica - feature/expect-actual-guillen

Documentación técnica: README.md y docs/S09_CAPACIDADES_NATIVAS.md.

7. Diferencias Android / iOS e interoperabilidad

Android usa Java NumberFormat y el selector de Intent.ACTION_SEND; como Koin entrega el Context de aplicación, el selector lleva FLAG_ACTIVITY_NEW_TASK. iOS usa Foundation NSNumberFormatter y presenta UIActivityViewController desde el controlador raíz. La firma común no cambia y las particularidades quedan confinadas en androidMain e iosMain.

La app Swift importa el framework Shared e invoca las funciones Kotlin exportadas desde archivos de nivel superior para iniciar Koin y presentar MainViewController. Este enlace existente se inspeccionó en el proyecto, pero su ejecución no se certifica desde Windows.

8. Conclusión

La solución incorpora las dos capacidades nativas solicitadas y mantiene el dominio y la interfaz compartidos. Las compilaciones y pruebas disponibles finalizaron correctamente. La validación funcional completa del entregable requiere ejecutar ambas aplicaciones y adjuntar las capturas faltantes; además, la conexión a PharmaSoft pertenece a una integración previa no presente en esta rama.

Fuente: Guía Práctica de Laboratorio N.º 09, 'Capacidades nativas con expect/actual en PharmaMobil', 06/10/2026; código y documentación de la rama indicada.